

Università degli Studi di Napoli Federico II

Corso di Laurea in Informatica

Gestore Ospedale e Turni Ospedalieri

Relazione di Progetto e Documentazione Software

Gruppo: 24

Pio Musco

Salvatore Gentile

Diego Maffettone

Repository GitHub del Progetto:

<https://github.com/00P2026/Gruppo24>

Anno Accademico 2025/2026

Indice

1	Introduzione e Requisiti di Sistema	2
1.1	Requisiti Funzionali Principali	2
2	Architettura Software: BCE e DAO Pattern	2
2.1	Organizzazione dei Package	2
2.2	Disaccoppiamento e Dependency Inversion	3
3	Modellazione UML e Scelte Progettuali	3
3.1	Modello del Dominio (Domain Model)	3
3.2	Class Diagram di Dettaglio	4
3.2.1	Scelte di Astrazione ed Elementi Omessi	4
4	Diagrammi di Sequenza	6
4.1	Flusso 1: Registrazione Nuova Prestazione (Medico)	6
4.2	Flusso 2: Registrazione Dimissione Paziente (Amministratore)	7
5	Manuale d'Uso dell'Applicativo	8
5.1	Accesso al Sistema	8
5.2	Guida per l'Amministratore	8
5.3	Guida per il Medico	8
6	Qualità del Codice, Documentazione e Consegna	9
6.1	Documentazione Interna (Javadoc)	9
6.2	Analisi della Qualità (SonarQube)	9
6.3	Risorse Repository GitHub	9

1. Introduzione e Requisiti di Sistema

Il presente documento costituisce la relazione finale di accompagnamento all'applicativo **Gestore Ospedale e Turni Ospedalieri**. Il sistema è stato concepito per automatizzare e monitorare le operazioni cliniche e logistiche all'interno di una struttura ospedaliera, garantendo la coerenza nell'allocazione dei posti letto e la corretta gestione dei turni del personale medico.

1.1 Requisiti Funzionali Principali

Il sistema gestisce due tipologie di utenti previa autenticazione con credenziali univoche:

- **Amministratori:**
 - Gestione anagrafica dei pazienti e registrazione dei ricoveri (con assegnazione di letti non sovrapposti temporalmente).
 - Consultazione dei pazienti in scadenza di dimissione e ricerca rapida dei letti liberi per reparto.
 - Monitoraggio dello stato di indisponibilità dei posti letto in tempo reale.
 - Gestione dei periodi di **Malattia** dei medici, con individuazione automatica delle scoperture (turni e prestazioni) e suggerimento dei medici sostitutivi idonei dello stesso reparto.
- **Medici:**
 - Consultazione della propria agenda giornaliera e settimanale dei turni.
 - Registrazione e aggiornamento dell'esito delle prestazioni erogate (visite o interventi chirurgici), previa verifica della propria presenza in turno e dell'assenza di sovrapposizioni temporali.

2. Architettura Software: BCE e DAO Pattern

L'applicazione rispetta il pattern architetturale **Boundary-Control-Entity (BCE)** integrato con il pattern **Data Access Object (DAO)** per disaccoppiare la logica di business dall'infrastruttura di persistenza.

2.1 Organizzazione dei Package

- **gui (Boundary):** Sviluppato in Java Swing, gestisce esclusivamente la visualizzazione e l'interazione con l'utente.
- **controller (Control):** Contiene un'unica classe coordinatrice (**Controller**) che orchestra le operazioni tra la GUI e lo strato di persistenza, mantenendo lo stato di sessione corrente.
- **model (Entity):** Raccoglie le entità di dominio e i <<record>> usati per il trasferimento dati immutabile (es. **TurnoScoperto**).
- **dao (Persistence Abstraction):** Definisce le interfacce Java contenenti i contratti dei servizi richiesti al database.

- **postgresdao (Concrete Infrastructure)**: Implementa le interfacce DAO interagendo con la base di dati tramite query SQL native e JDBC.
- **database**: Gestisce la connessione al database PostgreSQL.

2.2 Disaccoppiamento e Dependency Inversion

In conformità con il *Dependency Inversion Principle*, il **Controller** interagisce esclusivamente con le interfacce del package **dao**. La classe di controllo non possiede dipendenze verso l'infrastruttura concreta **postgresdao**, rendendo il DBMS sottostante un dettaglio implementativo facilmente sostituibile.

3. Modellazione UML e Scelte Progettuali

3.1 Modello del Dominio (Domain Model)

Il Modello del Dominio rappresenta i concetti della realtà ospedaliera liberati dai dettagli implementativi del database relazionale (es. Foreign Key rimosse e modellate come associazioni concettuali):

- **Reparto e Stanze**: Relazione di composizione (*Reparto ... Stanza*), poiché la stanza non ha vita autonoma fuori dal reparto.
- **Stanza e Letti**: Relazione di aggregazione (*Stanza ◇ ... Letto*), che modella la possibilità di riassegnare un posto letto.
- **Ricovero e Prestazioni**: Relazione di composizione tra il *Ricovero* e le *Prestazioni* erogate all'interno di quella pratica clinica.

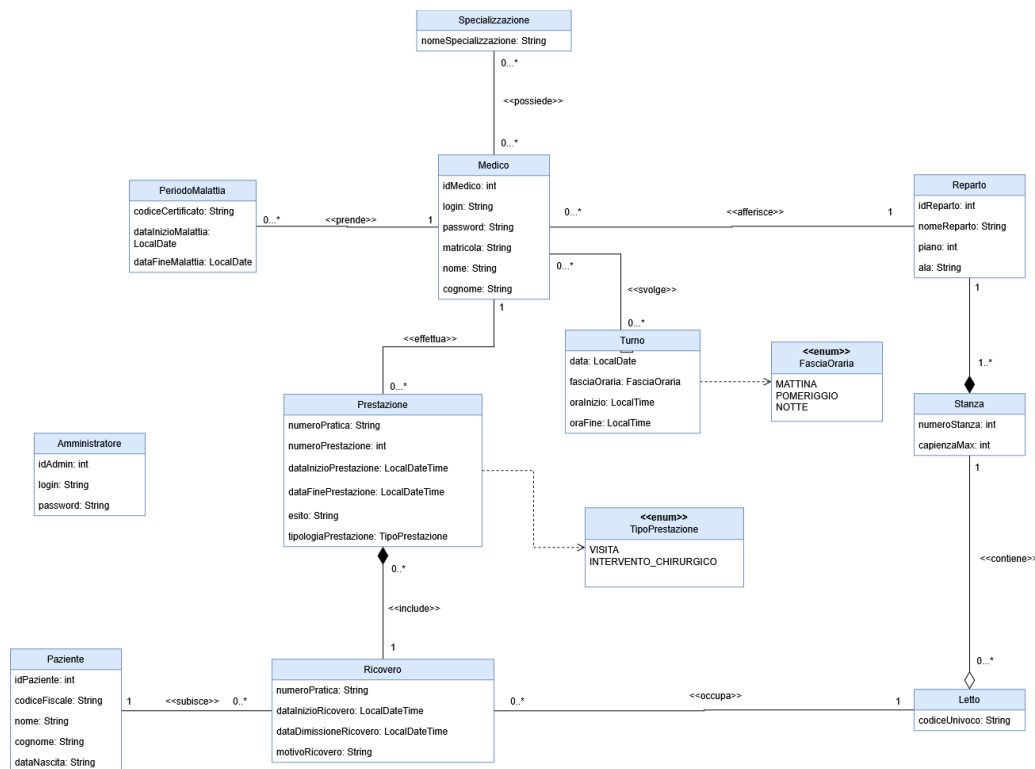


Figura 1: Modello del Dominio del Problema

3.2 Class Diagram di Dettaglio

Il diagramma di dettaglio mostra l'intera struttura del codice Java implementato. I punti chiave della progettazione sono:

- Distinzione visiva immediata tramite codifica cromatica (GUI in verde, Controller in blu, Model in rosso, Interfacce DAO in viola, Implementazioni Postgres in giallo).
- Utilizzo dei tipi `<<record>>` per la gestione dei dati aggregati (es. `TurnoScoperto`, `PrestazioneScoperta`).

3.2.1 Scelte di Astrazione ed Elementi Omessi

Per garantire un'elevata leggibilità del diagramma e allo scopo di descrivere principalmente l'architettura del sistema, sono state adottate le seguenti scelte di astrazione:

- **Boilerplate Code e Metodi private/utility (Getter, Setter, Costruttori e toString):** Gli accessori standard e i costruttori sono stati omessi dai box delle classi di dominio e del Controller. Nel diagramma sono riportati esclusivamente gli attributi d'istanza veri e i metodi recanti logica di business o firme di servizio.
- **Dettagli di Layout GUI, Event Listener e componenti Swing:** Gli specifici `ActionListener`, pannelli di supporto Swing e componenti grafici di basso livello sono stati omessi dal package `gui`, mantenendo unicamente le classi pannello principali e i relativi metodi di aggiornamento (`refresh`).
- **Associazioni dirette tra Controller e Model non di sessione:** Il Controller presenta linee di associazione diretta (linea continua) soltanto verso i tre Model che costituiscono lo stato di sessione attivo (`Amministratore`, `Medico`, `Reparto`). Per le restanti entità di dominio (`Paziente`, `Ricovero`, `Prestazione`, ecc.), la loro manipolazione avviene esclusivamente come parametri di passaggio o valori di ritorno nei metodi del Controller e dei DAO, evitando associazioni strutturali non veritiere.
- **Ridondanza delle relazioni PostgresDAO → Model:** Le linee di dipendenza d'uso verso il Model sono state tracciate partendo esclusivamente dalle Interfacce DAO. Poiché le classi concrete del package implementano tali interfacce, la loro dipendenza verso il Model è implicitamente garantita per ereditarietà del contratto.

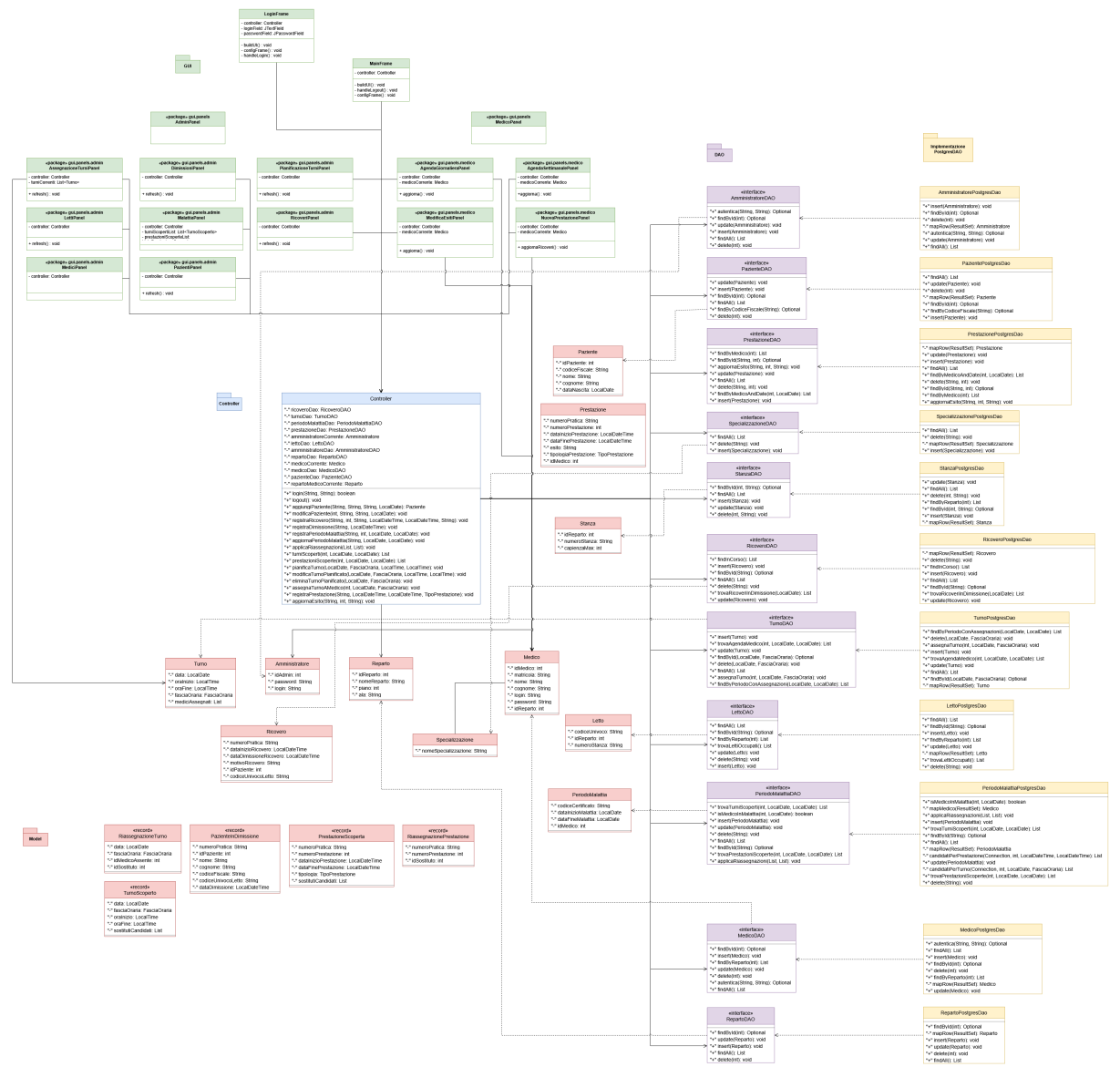


Figura 2: Class Diagram di dettaglio completo dell'Applicazione

4. Diagrammi di Sequenza

Di seguito vengono illustrati i flussi di esecuzione per due funzionalità chiave del sistema, evidenziando le interazioni trasversali tra i vari package.

4.1 Flusso 1: Registrazione Nuova Prestazione (Medico)

Il diagramma mostra l'iter con cui un medico inserisce una prestazione clinica. Il sistema esegue validazioni locali sulla GUI, controlli di dominio sul Controller (verifica presenza in turno e assenza stato di malattia tramite `PeriodoMalattiaDAO`), e infine la persistenza via `PrestazioneDAO`.

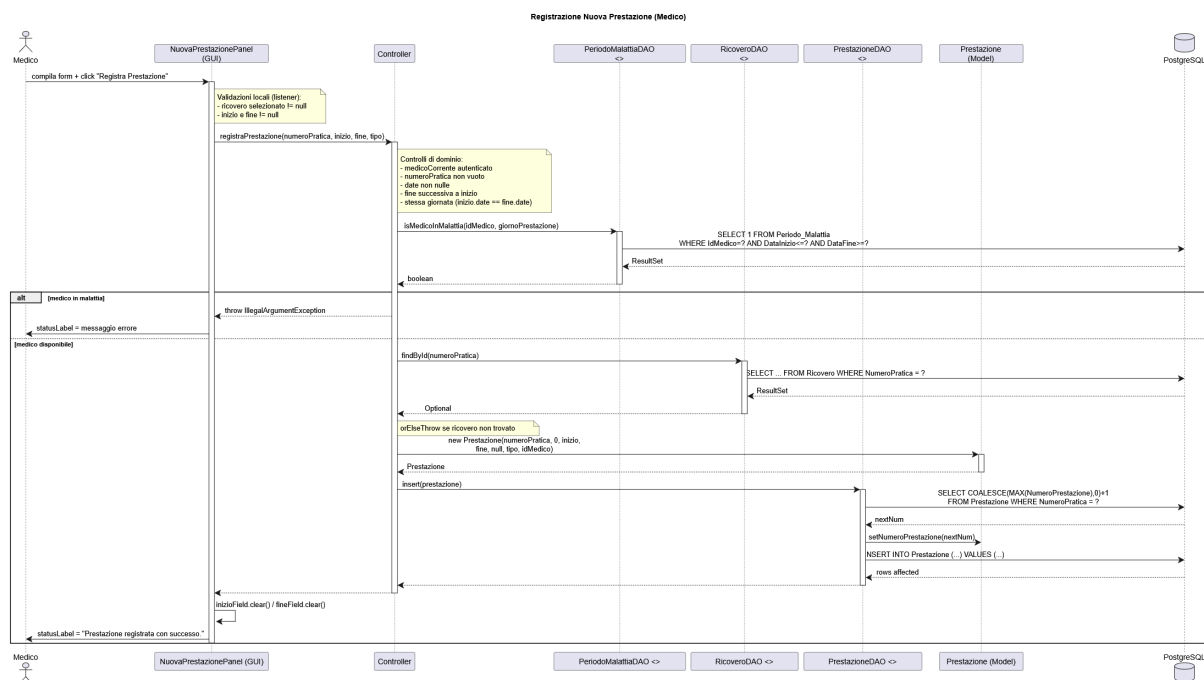


Figura 3: Diagramma di Sequenza: Registrazione Nuova Prestazione.

4.2 Flusso 2: Registrazione Dimissione Paziente (Amministratore)

Il diagramma illustra la gestione della dimissione di un ricovero attivo da parte dell'amministratore. Viene verificato lo stato `isInCorso()` della pratica sul Model prima dell'aggiornamento su `RicoveroDAO` e del conseguente refresh delle viste.

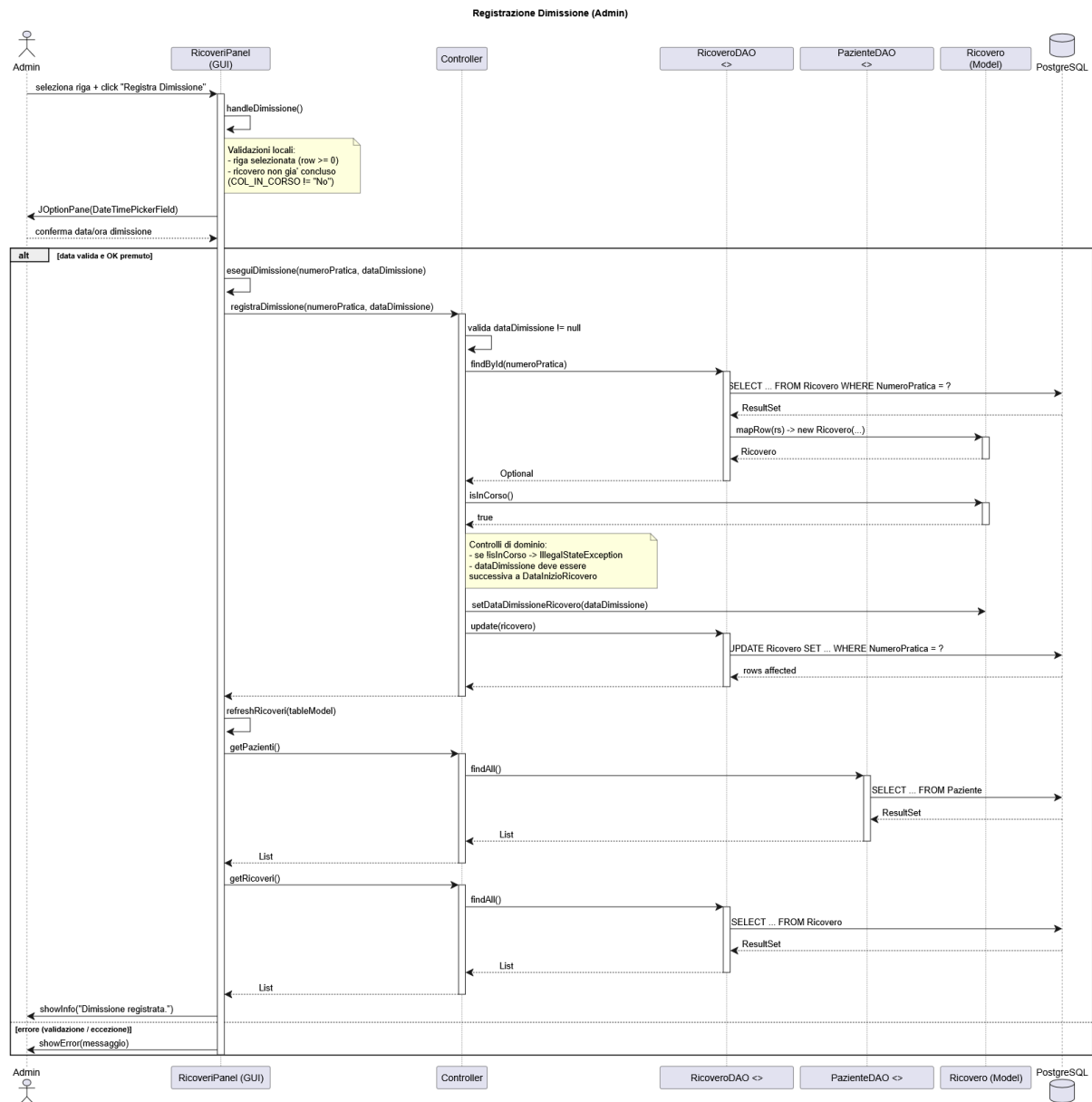


Figura 4: Diagramma di Sequenza: Registrazione Dimissione Paziente.

5. Manuale d'Uso dell'Applicativo

5.1 Accesso al Sistema

All'avvio dell'applicazione viene presentata la schermata di login. L'utente deve inserire le proprie credenziali univoche. Il sistema riconosce automaticamente il ruolo (Amministratore o Medico) e carica la relativa interfaccia grafica dedicata.

5.2 Guida per l'Amministratore

1. **Inserimento Pazienti:** L'inserimento di nuovi pazienti a DB avviene attraverso il panel Pazienti, la registrazione di eventuali ricoveri a loro relativi è possibile soltanto dopo aver completato il processo di inserimento iniziale.
2. **Gestione Ricoveri:** Dalla scheda dei ricoveri è possibile registrare un nuovo ricovero selezionando un paziente e un letto libero. I letti occupati vengono evidenziati con il codice in colore rosso.
3. **Registrazione Dimissioni:** Selezionare un ricovero in corso dalla tabella, impostare la data/ora di dimissione e confermare. Il letto verrà immediatamente liberato.
4. **Creazione ed Assegnazione Turni:** L'amministratore può creare nuovi turni specificando data, fascia oraria e assegnarvi medici non occupati tramite gli appositi form nei panel dedicati.
5. **Gestione Assenze e Malattie:**
 - Inserire il periodo di assenza per un medico specificando le date di inizio e fine.
 - Il sistema calcolerà e mostrerà automaticamente la lista dei turni e delle prestazioni che rimangono scoperte.
 - Verrà fornito un elenco di *medici sostitutivi proposti*, filtrati per medesimo reparto e disponibilità oraria per la riassegnazione rapida.

5.3 Guida per il Medico

1. **Consultazione Agenda:** Il medico può visualizzare la pianificazione dei propri turni lavorativi con vista giornaliera e settimanale.
2. **Inserimento Prestazione:**
 - Selezionare il ricovero del paziente su cui si è operato.
 - Inserire tipologia (Visita o Intervento) e intervallo orario. Il sistema accetta la registrazione solo se compresa in un turno attivo del medico e non sovrapposta ad altre prestazioni.
 - Aggiornare l'esito della prestazione direttamente dalla tabella.

6. Qualità del Codice, Documentazione e Consegna

6.1 Documentazione Interna (Javadoc)

Tutto il codice sorgente è stato integralmente documentato utilizzando gli standard Javadoc. Ogni interfaccia DAO, metodo del Controller e classe di modello contiene descrizioni dettagliate di parametri, valori di ritorno ed eccezioni sollevate.

6.2 Analisi della Qualità (SonarQube)

Il progetto è stato sottoposto ad analisi statica tramite SonarQube per garantire un'elevata manutenibilità:

- Assenza di *Code Smells* critici o bloccanti.
- Gestione corretta della chiusura delle risorse JDBC (`Connection`, `PreparedStatement`, `ResultSet`) tramite blocchi `try-with-resources`.
- Rispetto delle convenzioni di nomenclatura Java su tutti i package e classi.

6.3 Risorse Repository GitHub

L'intero codice sorgente è reperibile al seguente indirizzo:

<https://github.com/00P2026/Gruppo24>